Handwritten digit recognition with synthetic training examples

Can 120 synthetic variations of handwritten digits improve a classifier that already recognizes 295 of 300 held-out examples?

We continued the Assignment 1 (A1) digit-recognition project using its saved convolutional neural network (CNN) and the original MNIST split. The target application accepts a single digit from a mouse, a finger or an uploaded image. This experiment measures performance on MNIST; it leaves performance on new users' drawings open.



d0-v02 d2-v03 d4-v06 d5-v07 d7-v09 d9-v11

Figure 1. Six released synthetic training images. Each retains the form of a different MNIST training example. All examples in the released set are marked as synthetic.

What changed from A1

We used frozen TA-TiTok and MaskGen-L models to edit examples offline. A saved image, its source ID and its review decision travel together. The web demo runs the fine-tuned CNN and displays which part of the input contributed to its prediction.

What the experiment found

The augmented model recognized **296 of 300** test digits. Its accuracy was **98.67%** and its macro-F1 was **98.68%**. A matched run using only original training digits recognized **297**. One extra correct prediction over A1 gives limited evidence about the benefit of the synthetic images themselves.

Dataset and review

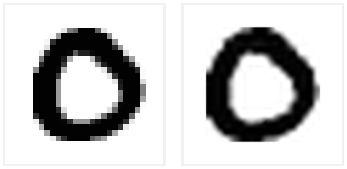
A1 contains 2,000 public MNIST images, balanced across digits 0-9. The manifest records the source partition, index, project split and pixel hash. A2 reused that split. Every synthetic image traces back to a distinct A1 training image [1].

Split	Original	Synthetic	Distinct inputs
Training	**1,400**	**120**	**1,520**
Validation	300	0	300
Test	300	0	300

Each class contributes 140 originals and 12 synthetic variants to training. Synthetic images make up **7.9%** of distinct training inputs (120/1,520). The 300 validation and 300 test images each contain 30 examples per class.

How the writing varies

Thin stroke



Original Synthetic

Thick stroke



Original Synthetic

Rotation



Original Synthetic

Figure 2. Each pair shows a training reference and its derivative. The set contains 40 examples in each style family, four per family for each digit.

Review and split integrity

After viewing the collection, the reviewer approved all 120 images in one batch and retained the source digit labels. The review CSV records that decision. The training script checks PNG hashes, source labels and train membership before loading them. Approval signifies a batch decision; it provides no independent rating of each digit's readability.

Editing the strokes of a known digit

TA-TiTok represents an image as an ordered sequence of visual tokens. MaskGen predicts masked tokens given text and the surrounding tokens. The original work supports discrete and continuous tokenizers and uses text when decoding. We used the public discrete (VQ) checkpoints for TA-TiTok BL128 and MaskGen-L [2-4].

A source-guided editing procedure

1. **Prepare**
   A cropped MNIST training digit on a white 256 x 256 canvas.
2. **Set the style**
   Thin or thicken the stroke, or rotate it by 8 or 14 degrees.
3. **Encode and refill**
   TA-TiTok encodes 128 tokens. MaskGen replaces 4-8 masked tokens.
4. **Decode and combine**
   Decode both sequences with the same text. Apply their difference near the stroke.

Figure 3. Project adaptation of the published models. The source determines the glyph's identity. The three style operations supply most of the visible variation.

Why we changed the generator input

In the first text-only run, prompts produced pens, paper and decorative marks. The frozen A1 classifier agreed with the requested digit for 14 of 120 images. With a source digit as a guide, it agreed for 113 of 120. These figures describe candidate quality on derived images; the held-out test scores appear on page 5.

For each masked position we sample from eight high-scoring alternative codes. The two decoded images isolate the change caused by MaskGen. We limit that difference to 64 grayscale levels and weight it by the source stroke and its narrow surrounding area. Across the release, the mean absolute change on this support was 2.28 out of 255 relative to the style-only image. The frozen A1 classifier gave the same top prediction for the style-only and final versions of all 120 images.

Training and model selection

Both fine-tuning runs started from the same saved A1 CNN. It has three convolutional blocks (32, 64 and 128 filters), max pooling and a fully connected classifier for ten digits. Input images pass through the original A1 preprocessing: invert dark ink, fit within 24 x 24 pixels and centre it on a black 32 x 32 canvas.

Run	First 56 slots per batch	Final 8 slots per batch
A1 baseline	Frozen checkpoint	No additional optimization
Real-only control	Original training images	Replayed originals
A2 with synthesis	**Same original images**	**Same labels, synthetic images**

Each epoch used all 1,400 originals once plus 200 additional draws. In the augmented run, 20 extra draws per class visit each of its 12 new examples at least once. Both branches used five epochs, seed 42, a batch size of 64 and Adam with learning rate 0.0001 and weight decay 0.0001.

Learning curves

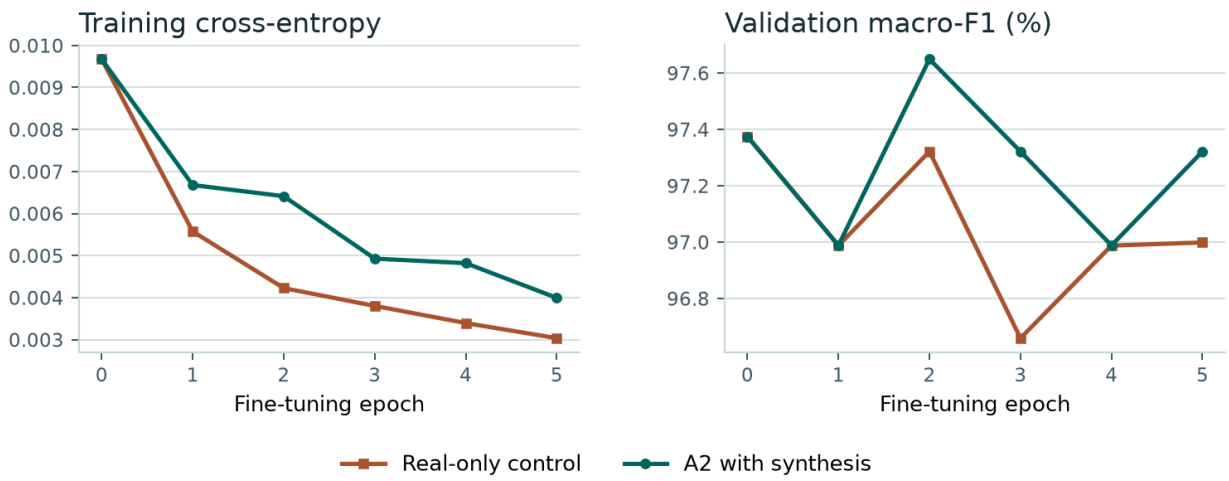


Figure 4. Epoch 0 is the original A1 checkpoint. Train loss is evaluated on the same 1,400 original images for both branches. Both fine-tuned branches reached their best validation macro-F1 at epoch 2.

We chose the checkpoint using validation macro-F1, with lower validation cross-entropy as the tie-breaker. The augmented model scored **97.65%** on validation, compared with **97.37%** for A1 and **97.32%** for the real-only control. Test results were computed after model and epoch selection.

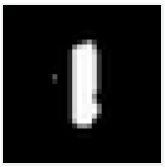
Held-out recognition performance

The evaluation used the same 300 test IDs, labels and preprocessing as A1. Accuracy is the fraction classified correctly. Macro-F1 averages the F1 of the ten digit classes with equal weight; each class's F1 balances precision and recall.

Model	Val. F1	Test correct	Test accuracy	Test F1
A1 baseline	97.37%	295/300	98.33%	98.35%
Real-only control	97.32%	297/300	99.00%	99.01%
A2 with synthesis	**97.65%**	**296/300**	**98.67%**	**98.68%**

Table 1. All F1 values are macro-averages. The control and augmented rows use the same architecture, number of updates, real-image order and extra label sequence.

Which predictions changed?



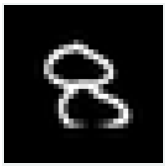
True 1

A1: 8 A2: 1



True 7

A1: 9 A2: 7



True 8

A1: 8 A2: 9

Figure 5. All three test cases whose predicted labels changed between A1 and A2. Both fine-tuned models fixed the 1 and the 7. Only the augmented branch introduced the 8-to-9 error.

What the comparison supports

Relative to A1, augmentation plus fine-tuning raised accuracy by **0.33 percentage points** and macro-F1 by **0.32 points**. The matched real-only control achieved the highest test score and made one fewer error than the augmented model. This run therefore does not establish an additional accuracy benefit from the 120 synthetic images.

A single digit changes accuracy by 0.33 percentage points on this test. We ran one seed and used a small, balanced MNIST subset. Generalization to phone touchscreens or mouse input would require a separately collected and labeled test set from users.

What the demo predicts and highlights

A user draws one digit or uploads an image. The local server prepares a 32 x 32 grayscale input, runs the selected CNN and returns a digit with its class scores. The page also shows the browser and device, average forward-pass time and a Grad-CAM overlay. For drawings, the server first crops to the visible stroke and pads the crop.

A correct prediction and a failure

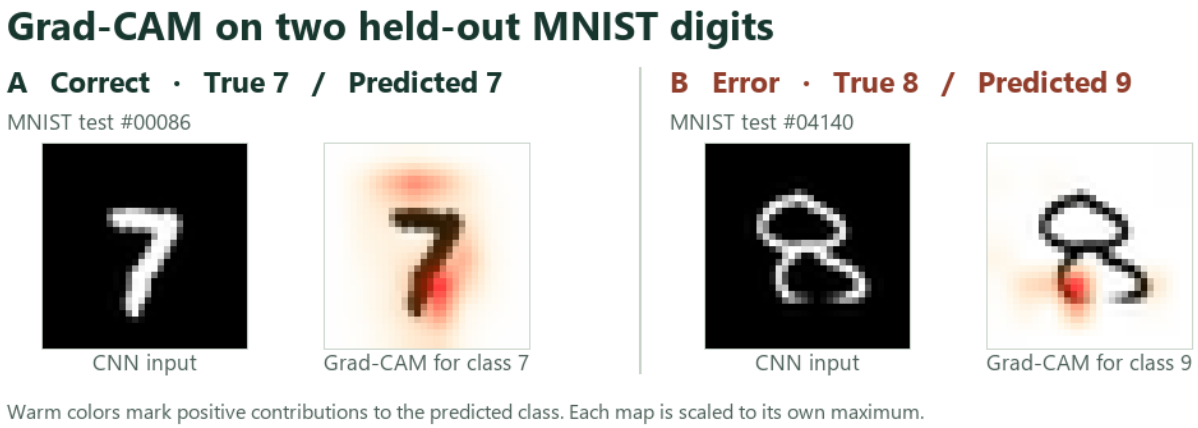


Figure 6. The deployed model's maps for two held-out inputs. Warm regions contributed positively to the predicted class. Each panel has its own color scale.

Grad-CAM weights the last convolutional feature maps by the average gradient of the predicted-class score, then highlights their positive contributions [5]. On the 7, the map covers parts of the top bar and descending stroke. On the 8, it highlights the middle-to-lower junction even though the label returned is 9. A heatmap localizes evidence for a decision; it cannot validate that decision.

Response time on the test computer

Measurement	Mean	Median	95th percentile
CNN forward pass	1.223 ms	1.094 ms	1.938 ms

We timed 100 distinct single-image inputs after 20 warm-up calls on an Intel Core i5-6500 CPU (Windows 10, PyTorch 2.6.0+cpu, two intra-op threads). The timer covers the encoder and classifier; preprocessing, Grad-CAM, network transfer and browser rendering add time outside that measurement.

The gallery shows precomputed synthetic images for the predicted digit. Users can inspect an image with its reference and record a training decision. New drawings are processed for inference; the demo does not add them to the dataset.

Running and interpreting the project

Reproduce the comparison

Clone the project and follow the Python environment setup in the root README. Python 3.12 and CPU PyTorch 2.6.0 were used for the published run. From the repository root, start the local app or repeat training with the accepted set.

```
python assignment2/demo.py
python assignment2/train_compare.py --pilot assignment2/outputs/reference_maskgen_cadcae3f15d2
```

Open the local app at 127.0.0.1:8766. The repository includes both classifier checkpoints, the accepted PNGs, review decisions, training history and predictions for each test ID. To generate a new set, open the numbered Colab notebook, inspect its ten-image preview, then enable the full batch. The published set was generated on CPU with the same recipe and pinned weights.

Ethics and limitations

The original handwriting comes from public MNIST. The synthetic derivatives carry its CC BY-SA 3.0 attribution, source hashes and modification history [1]. We keep uploaded drawings in memory for inference. A future study collecting them would need consent and a retention policy. The classifier always chooses among ten digits, so a large score on an unrelated mark can mislead a user. Touchscreen accuracy and writer-specific effects remain unmeasured.

References and project files

[1] Keras. MNIST digits classification dataset.
Data and attribution to Yann LeCun and Corinna Cortes, derived from NIST.

[2] Kim, D., et al. (2025). Democratizing Text-to-Image Masked Generative Models.
ICCV. The paper introduces compact text-aware one-dimensional tokens and MaskGen.

[3] Mozgovoy, K. (2025). ByteDance Unveil TA-TiTok Tokenizer. Neurohive.

[4] ByteDance. MaskGen release, implementation and checkpoint links.

[5] Selvaraju, R. R., et al. (2017). Grad-CAM. ICCV.

Project repository and local run guide